



LiteTest

Contributor Guide

Document Version: 1.0

Runner Version: 1.0.0

Test Version: 1.0.0

This document defines the contribution process, coding standards, documentation rules, and versioning guidelines for LiteTest.

Copyright (c) 2026 Paul Sinclair

License SPDX-License-Identifier: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Preface

This document is for LiteTest contributors who need guidance on enhancing and maintaining LiteTest.

For a list of other LiteTest documents and the repository layout, see the LiteTest Documentation Guide (`LiteTest_Documentation_Guide.md`).

For a glossary of terms, see the LiteTest Glossary Reference (`LiteTest_Glossary_Reference.md`).

A printer-friendly PDF file for this document is available in `docs/pdf/`.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The **Test** column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.
- Both Runner and Test use the version format "`M.m.p`" (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are published in a release without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction	1
2. Versioning Rules	2
3. Testing Requirements	3
4. Documentation Rules	4
5. Code Style	5
6. Writing Style Consistency	6
7. Pull Requests	7
8. Alternative Project Names and Rename Plan	8
Glossary	11

1. Introduction

This Contributor Guide defines the expectations and rules for contributing to LiteTest. It covers versioning, testing, documentation, code style, and pull request requirements.

Contributors should read this document before submitting changes to ensure consistency across the LiteTest codebase and documentation.

2. Versioning Rules

LiteTest uses semantic versioning: - For .h and .c files: **M.m.p** (major, minor, patch). - For .md files: **M.u** (major, update).

When to increment:

- Update:
 - Document updated.
- Patch:
 - Bug fixes or implementation improvements.
- Minor:
 - Minor additions to the APIs (e.g., new function or macro)
 - Minor additions to the LiteTest framework.
 - No breaking changes (syntax or behavior) to existing public APIs.
 - No breakind changes to the LiteTest framework
 - Minor refactoring of implementation.
- Major
 - Major additions to the LiteTest APIs.
 - Major additions to the LiteTest framework.
 - Significant refactoring of implementation.
 - The following require incrementing the Major version but should be avoided by having opt-in or other mechanism to maintain compatibility:
 - * Breaking changes (syntax or behavior) to existing public APIs.
 - * Breaking changes to the LiteTest framework.
 - If incremented, it must be updated for all files (.h, .c, and .md) to the same major value, even if the file was not otherwise modified or updated with update, minor, and patch reset to 0.

When updating the version for the Runner API: - Update `LT_RUNNER_VERSION` in `litetest_runner.h`. - Update `LT_RUNNER_VERSION_C` in `litetest_runner.c`.

When updating the version for the Test API: - Update `LT_TEST_VERSION` in `litetest_test.h`. - Update `LT_TEST_VERSION_C` in `litetest_test.c`.

When updating the version for a document: - Add a new entry to the top of the Document Version History table only when the document changes are being published in a release. - Increment `u` if the major version is not incremented. - Reset the update value to 0 when the major version is incremented.

API compatibility rules: - Public APIs are additive by default. - Do not change existing signatures. - Do not change return code meanings. - Deprecate before removing. - Provide migration guidance for major changes including how to opt-in for new features and behavior.

3. Testing Requirements

- Build and run tests from the repository root:

`make run`

- Ensure report formatting changes are reflected in documentation examples.
- Keep test code aligned with the current version of `litetest_runner.h`.

4. Documentation Rules

- `README.md` must remain short and onboarding-focused.
- A User Guide contains conceptual explanations and examples.
- A API Reference contains public API definitions only.
- An internal guide or reference must not leak into public docs.
- Update documentation when enhancing macros, behavior, or report format.

5. Code Style

- C99.
- POSIX.1-2001 APIs only.
- Keep `litetest_runner.h` self-contained.
- Keep `litetest_runner.c` implementation-only.
- Avoid intermixing test code with helper logic inside test group functions.

6. Writing Style Consistency

(See the full style rules in the Documentation Style Guide above.)

This section exists here to ensure contributors do not overlook the requirement for parallel structure and consistent writing patterns across all LiteTest documentation in all types of files (e.g., .md, .h, .c, .yaml, .sh, Makefile, etc.).

6.1 Documentation Style Guide

1. Tone

- Technical, precise, and neutral.
- No marketing language.
- Prefer clarity over cleverness.

2. Formatting

- Use backticks for code identifiers.
- Use fenced code blocks for file trees, examples, and commands.
- Keep line lengths reasonable for GitHub rendering.

3. Writing Guidelines

- Define terms once, and then use them consistently.
- Avoid synonyms for technical concepts (e.g., always “update version,” never “revision”).
- Keep paragraphs short.
- Use lists for enumerations.

4. Click to view

- Use **Click to view** sections and subsections to keep the document readable while still accommodating large amounts of technical detail:
 - Collapsing sections allows readers to scan the structure and expand only what they need.
 - This keeps the document manageable, avoids overwhelming readers with unrelated detail, and makes the document easier to navigate.

5. Writing Style Consistency To keep LiteTest documentation clear and easy to read, maintain **parallel structure** within lists and related sentences. In practice:

- Start list items with the same part of speech (typically a verb).
- Keep grammatical patterns consistent across bullets.
- Avoid mixing styles such as “Keep paragraphs short” with “Using lists for enumerations.”
- Rewrite items as needed so the list reads smoothly and uniformly.

This guideline applies to all LiteTest documentation (.md files).

7. Pull Requests

Before submitting a PR:

- Ensure tests pass.
- Update version numbers if needed.
- Update documentation as needed.
- Keep changes focused and well-scoped.

8. Alternative Project Names and Rename Plan

TODO: add OpemTest to table.

Preferred alternative names for LiteTest:

- CanaryRunner
- CanaryAssert
- CanaryProof
- CanaryTestRunner

These names are reserved as fallback options if a naming conflict requires a project rename.

Additional two-letter abbreviation candidates:

These notes are an informal naming screen only. They are based on how generic, descriptive, or commonly used the terms appear in software and testing. They are not a trademark search or legal clearance.

Name	Abbrev.	Informal conflict note	Likelihood
LiteTest	LT	Clear and close to the project's lightweight positioning, but both lite and test are common software terms, so overlap with existing package or tool names is plausible.	Medium
CanaryTest	CT	Strong fit for the canary theme, but canary and canary testing are already common software terms, which makes the name less distinctive.	Higher
CoreTest	CT	Clear and technical, but both core and test are common product words and may overlap with existing tools or internal packages.	Medium
ClearTest	CT	Readable and descriptive, but the name is broad and likely to overlap with existing testing or QA branding.	Medium
TinyTest	TT	Good match for a lightweight framework, but it is fairly descriptive and similar in shape to other small-test framework names.	Medium
TraceTest	TT	More distinctive than generic test compounds, though trace is still a common engineering term.	Medium-Low
SwiftTest	ST	Memorable, but Swift has strong existing association with Apple's Swift ecosystem, which could create confusion.	Higher
QuickTest	QT	Familiar and easy to say, but QuickTest has long-standing use in software testing and QA tooling.	Higher

Name	Abbrev.	Informal conflict note	Likelihood
CompactTest	CT	Fits the lightweight positioning and is somewhat more distinctive than CoreTest or ClearTest , though still descriptive.	Medium-Low
PublicTest	PT	Descriptive and understandable, but public is a common language and API term, which makes the name fairly broad.	Medium
APITest	AT	Directly describes API testing, but the term is highly generic and already widely used across tools, articles, and packages.	Higher
FastTest	FT	Strong performance-oriented signal, but fast and test are both generic terms and likely to overlap with existing tooling names.	Medium-High

Working preference among the two-letter candidates:

- **CompactTest** (CT) for a lower-conflict descriptive option.
- **TraceTest** (TT) for a more distinctive technical option.
- **CanaryTest** (CT) only if the canary theme is more important than name distinctiveness.

Fast low-risk rename plan:

1. Branch and freeze scope
 - Create a dedicated rename branch.
 - Restrict the change to naming and branding only.
2. Choose one approved replacement name
 - Select one of the four preferred alternatives.
 - Use the chosen name consistently across code, docs, and assets.
3. Update user-facing names first
 - Update the project title in **README.md**.
 - Update top-level headings in LiteTest documentation files.
 - Add a temporary note in **README.md** that the project was renamed from LiteTest.
4. Update branding assets
 - Duplicate existing branding files to the new naming scheme.
 - Regenerate PNG files from SVG files.
 - Keep old logo files temporarily for one release to avoid broken references.
5. Update generator and file references
 - Update **scripts/genpdf** to use the new document branding file names.
 - Update markdown image references to point to renamed branding files.

- Regenerate all PDFs and verify output paths.
6. Update source identifiers carefully
 - Update only external identifiers that represent the project brand.
 - Avoid changing public API symbols unless explicitly required.
 - If API symbol changes are required, treat them as a major-version change.
 7. Validate and test
 - Run `make run`.
 - Run full PDF generation and verify branding and title pages.
 - Verify no stale references remain with a repository-wide text search.
 8. Ship with rollback safety
 - Commit the rename in one focused commit.
 - Keep a rollback commit point before removing old compatibility references.
 - In the next release, remove temporary compatibility references after verification.

Glossary

For a glossary of general LiteTest terms, see the LiteTest Glossary Reference document. For a glossary of terms, see the LiteTest Glossary Reference (`LiteTest_Glossary_Reference.md`).

Contributor-Specific Terms:

- **Approver:** A contributor with commit access who reviews pull requests, enforces versioning rules, and ensures documentation consistency.
- **Breaking Change:** A change that alters public API behavior, removes or renames macros, changes return semantics, or requires a major version bump.
- **Contributor:** A person submitting code, documentation, fixes, or improvements to LiteTest.
- **Deprecation:** A public API element marked for removal in a future major version, requiring documentation updates and migration guidance.
- **Documentation Update:** Any change to released LiteTest documentation requires incrementing the update version or, if an API major version is incremented, incrementing the major version and resetting the update version to 0.
- **Internal API Change:** A change to an API's internals that does not affect public API users but may require updates to an Internal Guide or Internal Reference.
- **Major Increment:** A structural or conceptual overhaul of LiteTest that causes a breaking change.
- **Public API Change:** Any modification to the LiteTest Runner or Test API that requires a version bump.
- **Pull Request Scope:** A guideline requiring PRs to be focused, minimal, logically grouped, and not mixing unrelated changes.
- **Test Coverage Requirement:** The expectation that all changes to LiteTest include new tests (if adding behavior), updated tests (if modifying behavior), and no regressions.